

ALGORITHM PERFORMANCE MEASUREMENT AND BENCHMARKING TOOL

Lab Assignment 2 — Final Report

K.R. MANGALAM UNIVERSITY

School of Engineering & Technology | Department of SOET

Student Name	Harshit Kumar
Roll No.	2401201057
Program	BCA (AI & DS)
Semester	V
Course Code	ENCA351
Course Name	Design and Analysis of Algorithms Lab
Faculty	Dr. Aarti
Session	2026–27
GitHub Repository	Harshitkumar0018/Algorithm-Performance-Tool

Integrated report contents

Experiment Analysis • Benchmark Results & Graphs • Complexity Analysis & Conclusion • Streamlit
Output Evidence

1. Introduction

Algorithm analysis explains how the computational cost of a program changes as the input size grows. Two programs can produce the same result but have very different running times or memory requirements. This project turns that idea into an interactive experiment by measuring execution time, memory usage and operation counts for search algorithms and common program structures.

The submitted project is a Streamlit-based performance measurement tool. It separates the interface from the benchmark logic through four main files: **app.py**, **search_analysis.py**, **code_benchmark.py** and **benchmark_engine.py**. The existing report specifies **time.perf_counter** for timing and **tracemalloc** for peak memory measurement.

1.1 Importance of Measurement

- Big-O describes growth but does not expose all practical constants and language overhead.
- Timing shows the actual cost of an implementation in the selected execution environment.
- Memory profiling reveals space overhead that cannot be inferred from runtime alone.
- Operation counts make the amount of algorithmic work easier to compare.
- Testing several input sizes makes scalability visible instead of relying on a single small example.

1.2 System Workflow

The application follows a repeatable pipeline: select an algorithm → provide an input size → generate the required input → execute the selected function through the measurement engine → collect time, memory and operations → display the result. Automated benchmarking repeats this process over predefined sizes and creates tables and graphs.

Module	Main responsibility
app.py	Streamlit interface, tabs, controls, result metrics and plots.
search_analysis.py	Dataset generation, Linear Search and Binary Search.
code_benchmark.py	Single Loop, Nested Loop, Recursive Factorial and Iterative Factorial.
benchmark_engine.py	Repeated timing, peak-memory measurement, automation and growth-slope calculation.

1.3 Measurement Metrics

Execution time is the measured duration of the function. **Peak memory** is the maximum traced allocation observed during the memory run. **Operations** are algorithm-specific counts such as comparisons, loop iterations or recursive calls. Looking at all three together gives a more complete performance picture.

2. Core Implementation — app.py

The main application creates four functional areas: Run Single, Search Analysis, Code Benchmark and Automated Benchmark. The code is intentionally shown in a larger monospaced font so that it remains readable in the printed report.

A. app.py — Streamlit application

```
import pandas as pd
import streamlit as st
import benchmark_engine as be
import code_benchmark as cb
import search_analysis as sa

st.set_page_config(
    page_title="Algorithm Performance Tool",
    layout="wide"
)

st.title("Algorithm Performance Measurement and Benchmarking Tool")

tab1, tab2, tab3, tab4 = st.tabs([
    "Run Single", "Search Analysis",
    "Code Benchmark", "Automated Benchmark"
])

with tab1:
    options = {**sa.ALGORITHMS, **cb.SNIPPETS}
    name = st.selectbox("Algorithm / snippet", list(options))
    n = st.number_input("Input size", 1, 3000, 1000)

    if st.button("Execute"):
        if name in sa.ALGORITHMS:
            data = sa.generate_dataset(int(n))
            args = (data, data[-1])
        else:
            args = (int(n),)

        result, elapsed, memory = be.measure(
            options[name], *args
        )

        c1, c2, c3 = st.columns(3)
        c1.metric("Time (ms)", f"{elapsed*1000:.4f}")
        c2.metric("Peak memory (KB)", f"{memory:.2f}")
        c3.metric("Operations", result[1])

with tab4:
    if st.button("Run full benchmark"):
        st.session_state["results"] = be.run_all()

    results = st.session_state.get("results")
    if results:
        for category, df in results.items():
            st.dataframe(df.round(4), width="stretch")
```

```
        st.pyplot(  
            be.plot_comparison(  
                df, *be.CHARTS[category]  
            )  
        )  
    st.dataframe(  
        be.complexity_table(results),  
        width="stretch"  
    )
```

The interface delegates actual computation to the other modules. This keeps the user-interface layer small and allows the same benchmark functions to be reused by automated experiments.

3. Search Analysis Module

The search module creates a reproducible sorted dataset and implements two different search strategies. Linear Search scans from the beginning, while Binary Search checks the middle and reduces the remaining interval after each comparison.

B. `search_analysis.py` — Search algorithms

```
import random

def generate_dataset(n, seed=42):
    random.seed(seed)
    return sorted(
        random.sample(range(n * 10), n)
    )

def linear_search(data, key):
    comparisons = 0

    for i, value in enumerate(data):
        comparisons += 1
        if value == key:
            return i, comparisons

    return -1, comparisons

def binary_search(data, key):
    low = 0
    high = len(data) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if data[mid] == key:
            return mid, comparisons

        if data[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1, comparisons
```

For the reported automated experiment, the search key is selected as the last element. This gives Linear Search a full scan and provides a difficult comparison case while Binary Search continues to use midpoint reduction.

4. Code Benchmark Module

The code benchmark contains two loop structures and two factorial implementations. The functions return both a result and an operation count so the interface can show more than elapsed time alone.

C. `code_benchmark.py` — Benchmark snippets

```
def single_loop(n):
```

```

total = 0
for i in range(n):
    total += i
return total, n

def nested_loop(n):
    total = 0
    for i in range(n):
        for j in range(n):
            total += 1
    return total, n * n

def recursive_factorial(n):
    ops = 0

    def fact(k):
        nonlocal ops
        ops += 1

        if k <= 1:
            return 1

        return k * fact(k - 1)

    return fact(n), ops

def iterative_factorial(n):
    result = 1
    ops = 0

    for i in range(2, n + 1):
        result *= i
        ops += 1

    return result, ops

```

5. Benchmark Engine and Automation

The benchmark engine is the common measurement layer. It repeats timing measurements and separately traces memory. The automated functions group experiments into Search, Factorial and Loop categories. The growth-slope calculation provides an empirical indicator of how the measured time changes with input size.

D. benchmark_engine.py — Measurement and automation

```
import time
import tracemalloc
import numpy as np

def measure(func, *args, repeats=5):
    start = time.perf_counter()

    for _ in range(repeats):
        result = func(*args)

    avg_time = (
        time.perf_counter() - start
    ) / repeats

    tracemalloc.start()
    func(*args)
    peak = tracemalloc.get_traced_memory()[1]
    tracemalloc.stop()

    return result, avg_time, peak / 1024

def run_all():
    return {
        "Search": run_search_benchmarks(),
        "Factorial": run_factorial_benchmarks(),
        "Loops": run_loop_benchmarks()
    }

def growth_slope(group):
    return float(np.polyfit(
        np.log(group["Size"]),
        np.log(group["Time_ms"]), 1
    )[0])
```

The log-log slope should be treated as supporting experimental evidence rather than a replacement for theoretical complexity analysis. A slope near 1 supports approximately linear growth, while a slope near 2 supports approximately quadratic growth over the tested range.

6. Experimental Design

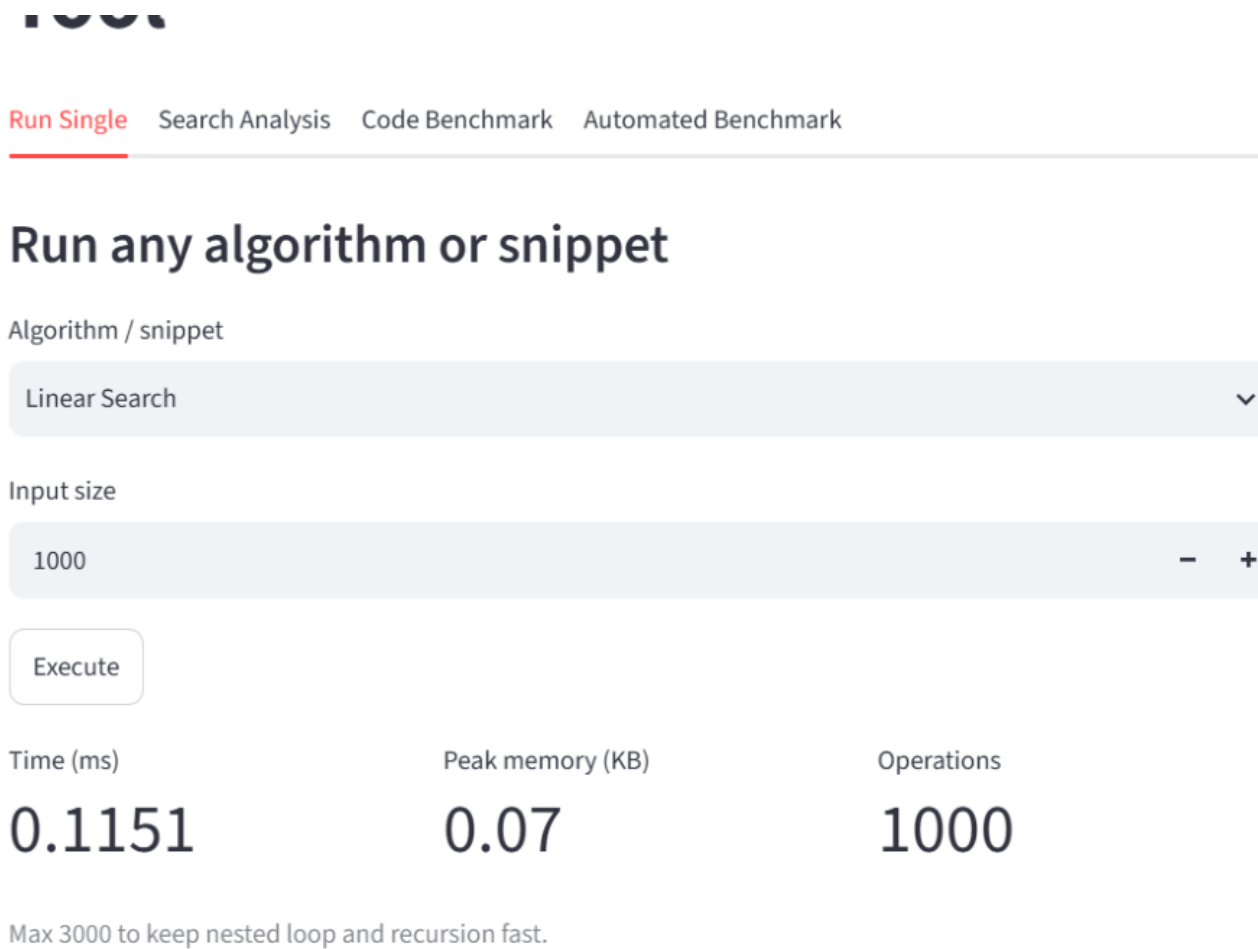
- The reported search sizes are 100, 1,000, 10,000 and 50,000 elements.
- The reported factorial sizes are 100, 500 and 1,000.
- Single-loop sizes extend to 50,000 iterations.

- The nested loop is limited to 5,000 because n^2 work reaches 25 million operations at that size.
- The experiment therefore focuses on showing growth trends while avoiding an unnecessarily expensive 2.5-billion-operation nested-loop run at $n=50,000$.

Part 2 contains the benchmark tables and visualization interpretation. Part 3 connects the measurements with theoretical complexity, trade-offs and final conclusions.

Output Screenshots — Streamlit Application

Figure A1 — Run Single Output



Actual Streamlit output: Linear Search with input size 1000.

Output Screenshots — Streamlit Application

Figure A2 — Search Analysis Output

Linear vs Binary Search

Dataset size

10000

− +

Search key

50956

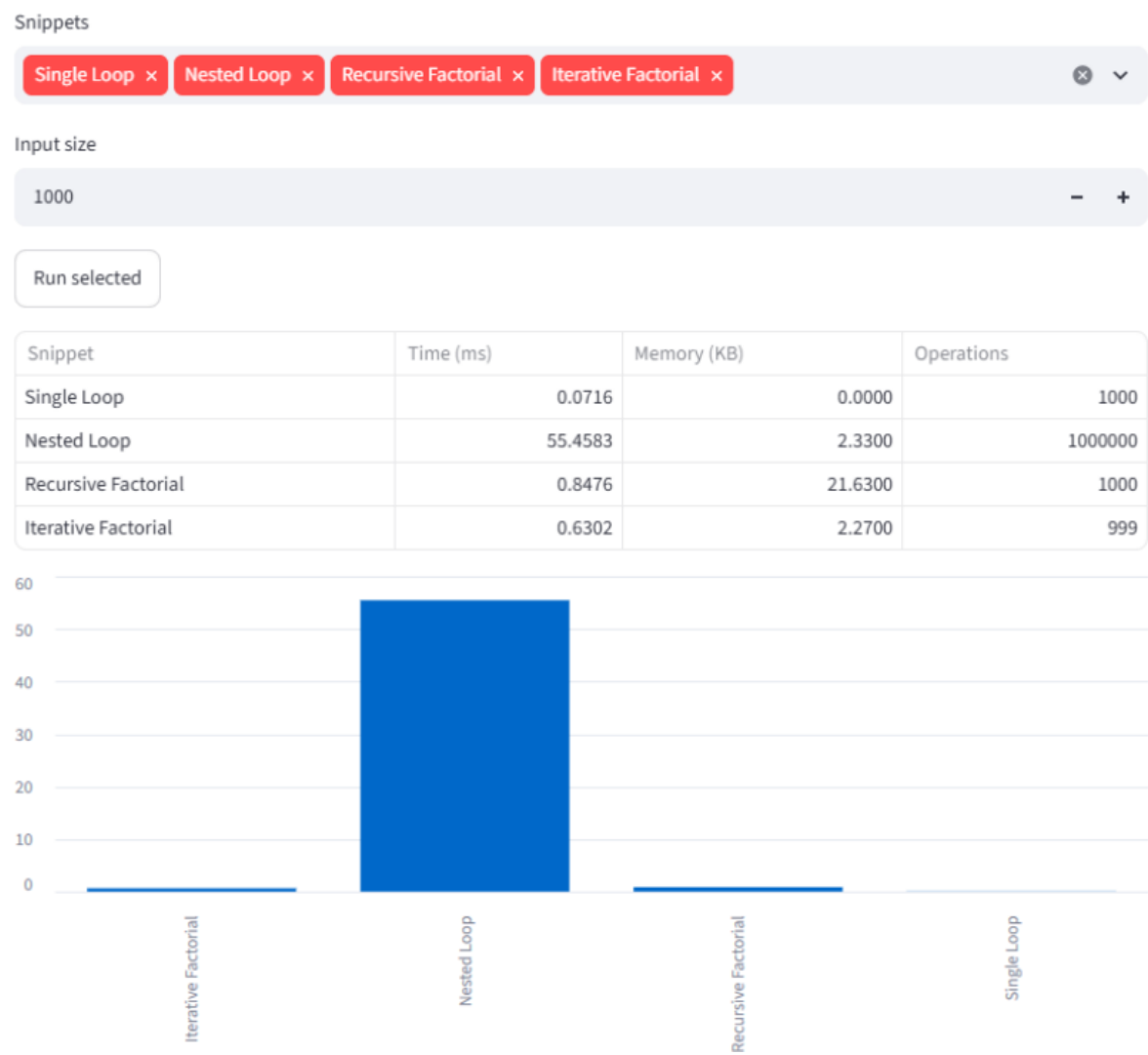
− +

Dataset: sorted, 10000 unique integers (min 12, max 99997)

Search

Output Screenshots — Streamlit Application

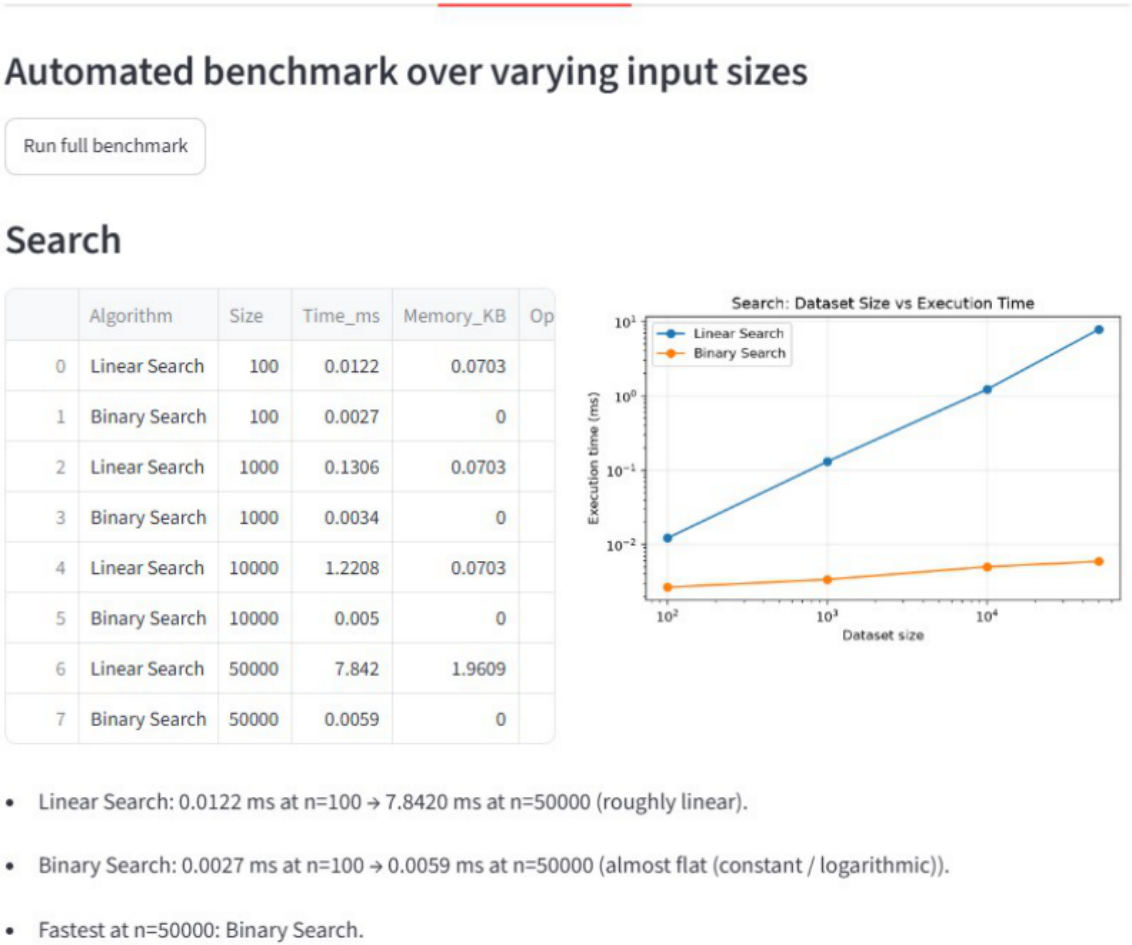
Figure A3 — Code Benchmark Output



Actual Streamlit output: comparison of the four benchmark snippets.

Output Screenshots — Streamlit Application

Figure A4 — Automated Benchmark Output



Algorithm Performance Measurement and Benchmarking Tool

Lab Assignment 2 — Part 2: Benchmark Results & Graphs

Experimental benchmark results, performance graphs and output summary

1. Search Benchmark

The search experiment uses the last element as the key, giving Linear Search a full scan while Binary Search repeatedly halves the remaining sorted range.

Algorithm	Size	Time (ms)	Memory (KB)	Operations
Linear Search	100	0.0289	0.1172	100
Binary Search	100	0.0084	0.0000	7
Linear Search	1,000	0.7311	0.2070	1,000
Binary Search	1,000	0.0100	0.1562	10
Linear Search	10,000	5.3986	0.2070	10,000
Binary Search	10,000	0.0151	0.1562	14
Linear Search	50,000	29.1193	0.2070	50,000
Binary Search	50,000	0.0100	0.1562	16

Search Performance Graph

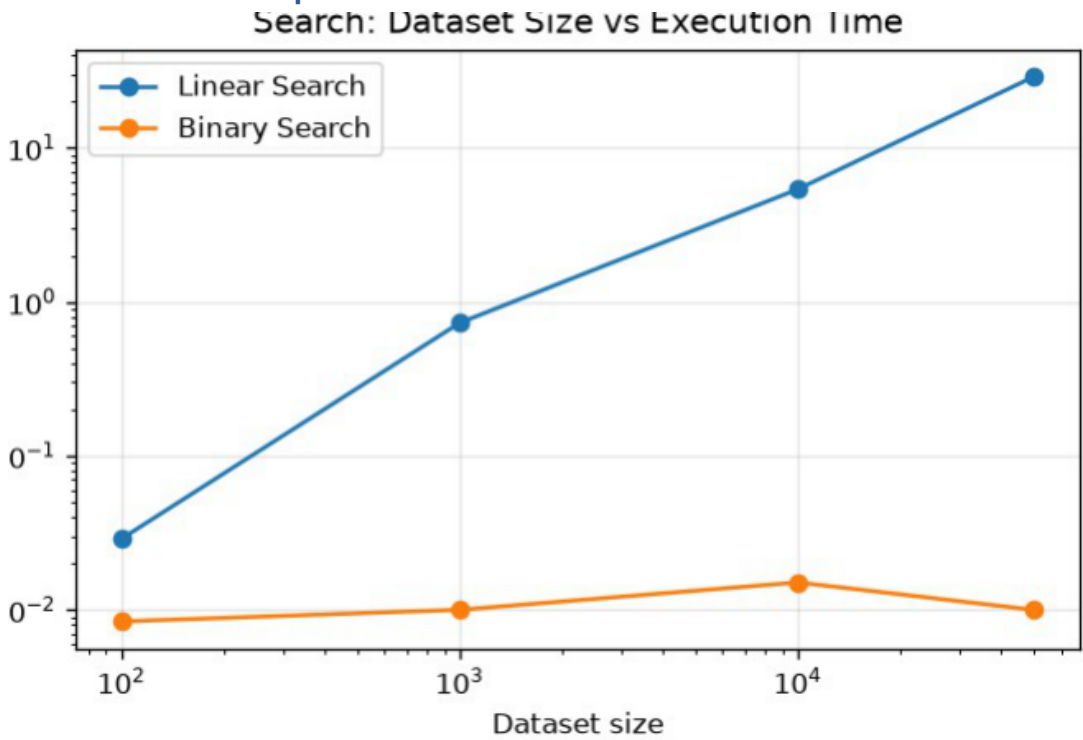


Figure 1. Dataset Size vs Execution Time — Linear Search vs Binary Search.

Observation: Linear Search rises strongly with input size, while Binary Search remains nearly flat over the tested range.

2. Factorial Benchmark

The factorial experiment compares recursive and iterative implementations for $n = 100, 500$ and $1,000$. Both have $O(n)$ algorithmic step growth, but recursion uses additional call-stack memory.

Algorithm	Size	Time (ms)	Memory (KB)	Operations
Recursive Factorial	100	0.0720	0.4297	100
Iterative Factorial	100	0.0443	0.2266	99
Recursive Factorial	500	0.7911	8.3555	500
Iterative Factorial	500	0.3536	1.1289	499
Recursive Factorial	1,000	1.8343	23.9805	1,000
Iterative Factorial	1,000	2.3670	2.3672	999

Factorial Performance Graph

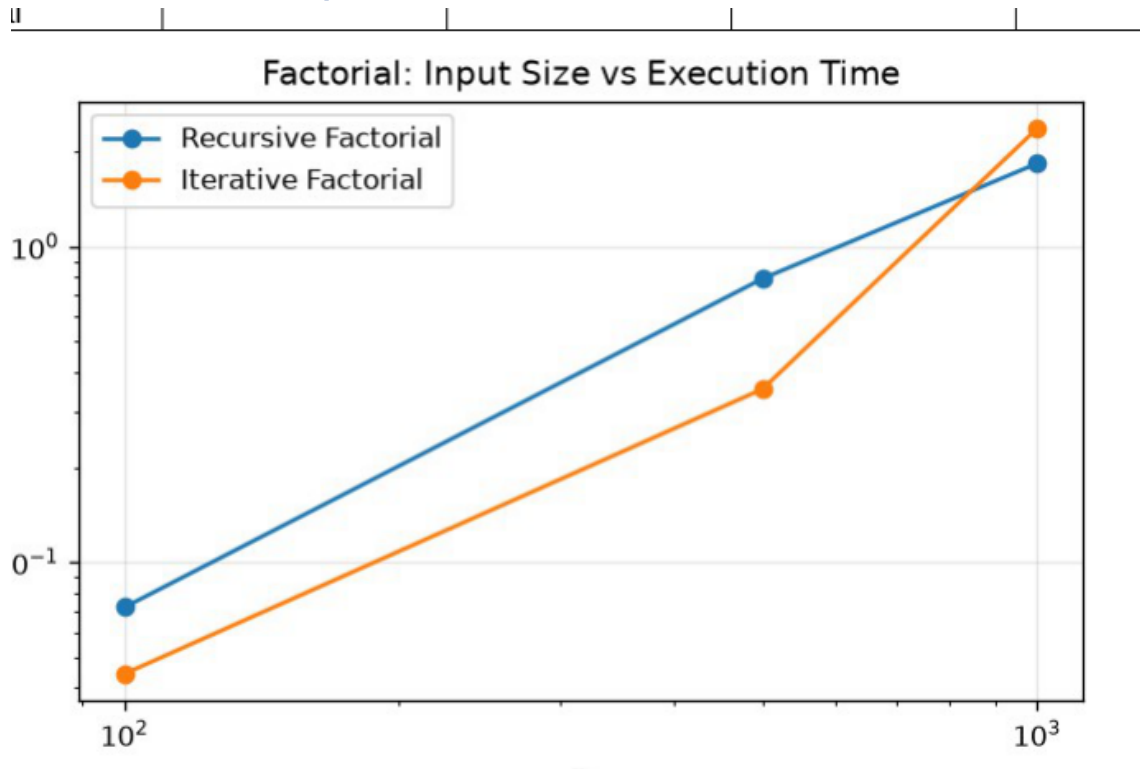


Figure 2. Input Size vs Execution Time — Recursive Factorial vs Iterative Factorial.

Observation: Both implementations increase with input size. The recursive version shows substantially higher reported memory usage at $n = 1,000$.

3. Loop Benchmark

The single loop performs n operations, while the nested loop performs n^2 operations. The nested benchmark is limited to 5,000 because 50,000 would imply 2.5 billion inner operations.

Algorithm	Size	Time (ms)	Memory (KB)	Operations
Single Loop	100	0.0213	0.0938	100
Single Loop	1,000	0.3346	0.1250	1,000
Single Loop	5,000	1.1830	0.1250	5,000
Single Loop	10,000	2.9256	0.1250	10,000
Single Loop	50,000	13.8235	0.1328	50,000
Nested Loop	100	1.8195	0.1406	10,000
Nested Loop	500	68.7639	0.2344	250,000
Nested Loop	1,000	339.7817	0.2344	1,000,000
Nested Loop	2,000	1,090.0080	0.2344	4,000,000
Nested Loop	5,000	5,919.1506	0.2344	25,000,000

Loop Performance Graph

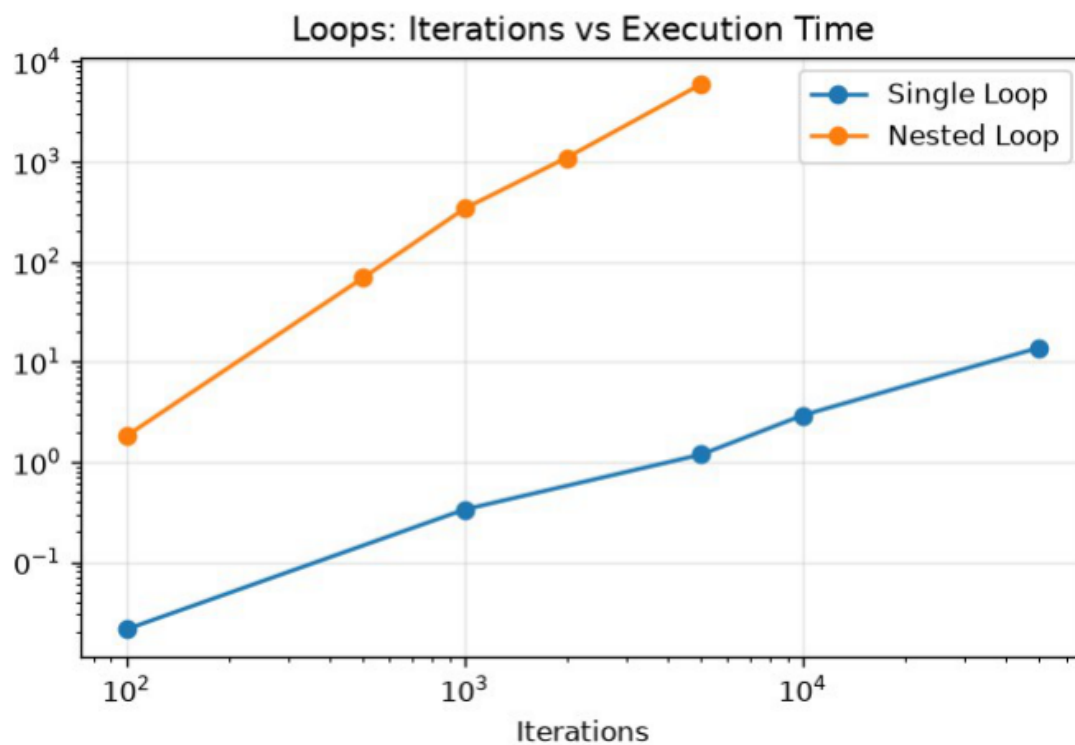


Figure 3. Number of Iterations vs Execution Time — Single Loop vs Nested Loop.

Observation: The nested loop grows dramatically faster, reaching 5,919.1506 ms at $n = 5,000$, while the single loop remains much lower.

4. Performance Trend Analysis

Category	Input Range	Observed Trend
Search	100 → 50,000	Linear Search increases strongly; Binary Search changes very little.
Factorial	100 → 1,000	Both increase with n ; recursion has higher memory usage.
Single Loop	100 → 50,000	Approximately linear growth.
Nested Loop	100 → 5,000	Rapid approximately quadratic growth.

The three graphs are placed directly after their respective benchmark tables so that the numerical results and visual growth trends can be compared together.

4.1 Search Graph Interpretation

At 50,000 elements, Linear Search is reported at 29.1193 ms with 50,000 comparisons, whereas Binary Search is reported at 0.0100 ms with 16 comparisons. The graph therefore visually supports the difference between $O(n)$ and $O(\log n)$ search behaviour on sorted data.

4.2 Factorial Graph Interpretation

The factorial curves are closer because both implementations perform a linear number of multiplication steps. The memory results provide the clearer implementation-level distinction because recursive calls add stack frames.

4.3 Loop Graph Interpretation

The loop graph shows the strongest divergence. The nested loop reaches 25 million operations at $n = 5,000$, while the single loop performs only 5,000 operations at the same input size.

5. Benchmark Output Summary

The following comparison summarizes the largest tested input for each category.

Implementation	Largest tested size	Time (ms)	Memory (KB)	Operations
Linear Search	50,000	29.1193	0.2070	50,000
Binary Search	50,000	0.0100	0.1562	16
Recursive Factorial	1,000	1.8343	23.9805	1,000
Iterative Factorial	1,000	2.3670	2.3672	999
Single Loop	50,000	13.8235	0.1328	50,000
Nested Loop	5,000	5,919.1506	0.2344	25,000,000

The measured values should be interpreted as experimental observations from the selected environment. Small timing values can be sensitive to system load and measurement overhead, whereas the operation counts and overall growth trends provide more stable evidence for the algorithmic comparison.

6. Key Findings

- **Binary Search** is the fastest search method in the reported sorted-data experiment and requires far fewer comparisons.
- **Iterative Factorial** has a much smaller auxiliary-memory requirement than the recursive implementation.
- **Single Loop** scales much better than the nested loop because its work grows linearly rather than quadratically.
- The benchmark graphs confirm that input size has an increasing impact on algorithms with higher growth rates.
- The nested-loop benchmark is intentionally capped at 5,000 because 50,000 would imply 2.5 billion inner operations.

The next part of the assignment uses these experimental results for the theoretical-versus-observed complexity analysis, time-space trade-offs and final conclusion.

Algorithm Performance Measurement and Benchmarking Tool

Lab Assignment 2 — Part 3: Complexity Analysis & Conclusion

Theoretical Complexity, Observed Growth, Trade-offs and Final Findings

1. Theoretical vs Observed Complexity

Big-O notation describes dominant growth. Experimental measurement adds practical effects such as implementation overhead, memory allocation and Python's handling of increasingly large integer values. The table below reproduces the complexity and observed-growth information from the submitted report.

Algorithm	Theory	Largest	Time ms	Memory KB	Slope	Observed
Linear Search	$O(n)$	50,000	29.1193	0.21	1.09	Linear
Binary Search	$O(\log n)$	50,000	0.0100	0.16	0.05	Almost flat
Recursive Factorial	$O(n)$	1,000	1.8343	23.98	1.42	Between linear and quadratic
Iterative Factorial	$O(n)$	1,000	2.3670	2.37	1.65	Between linear and quadratic
Single Loop	$O(n)$	50,000	13.8235	0.13	1.03	Roughly linear
Nested Loop	$O(n^2)$	5,000	5919.1506	0.23	2.07	Roughly quadratic

The log-log slope is an empirical indicator. A value close to 1 supports approximately linear growth, a value close to 2 supports approximately quadratic growth, and a value near 0 indicates little change over the measured range. It is evidence for the trend, not a replacement for theoretical analysis.

2. Search Algorithm Analysis

At 50,000 elements, Linear Search took 29.1193 ms and made 50,000 comparisons. Binary Search took 0.0100 ms and made 16 comparisons. Based on the reported values, Binary Search is about 2,912 times faster in that particular run. The more important conclusion is the growth pattern: Linear Search scales directly with n , whereas Binary Search repeatedly halves the remaining interval.

Binary Search has an important prerequisite: the data must be sorted. Therefore, the speed advantage is most useful when the dataset is already sorted or when many searches justify the cost of maintaining sorted data.

3. Recursive vs Iterative Factorial

For $n=1,000$, the reported recursive version took 1.8343 ms and used 23.9805 KB, while the iterative version took 2.3670 ms and used 2.3672 KB. Both use a linear number of multiplication steps in the implemented model, but recursion stores an active chain of calls, giving $O(n)$ auxiliary stack space compared with $O(1)$ extra space for iteration.

The timing difference should not be overinterpreted. Both versions are in the same asymptotic time class, and Python's integer arithmetic becomes more expensive as factorial values grow. The stronger practical result is the substantially lower memory requirement of iteration.

4. Single Loop vs Nested Loop

A single loop performs one operation per input position, giving $O(n)$ time. A nested loop executes the inner loop for each outer-loop iteration, giving $O(n^2)$ time. At $n=5,000$ the reported single loop takes 1.183 ms for 5,000 operations, while the nested loop takes 5,919.1506 ms for 25 million operations.

This makes the nested implementation roughly 5,004 times slower in the reported experiment. The result demonstrates that changing the growth class can dominate all small constant-factor differences once the input becomes sufficiently large.

5. Complexity Observations

- Linear Search and Single Loop have slopes close to 1, supporting approximately linear growth.
- Nested Loop has a slope of about 2.07, supporting quadratic behaviour.
- Binary Search has a very small measured slope because logarithmic growth changes slowly across the tested range.
- Factorial slopes are above 1 even though the algorithmic step count is $O(n)$, because Python integer values become larger as n increases.
- Very small timings such as Binary Search near 0.01 ms are sensitive to measurement noise, so repeated measurements are preferable.
- Search and loop functions use very little additional memory because they do not create large auxiliary structures.

6. Time–Space Trade-offs

Implementation	Strength	Trade-off / condition
Binary Search	Fast lookup on large sorted data	Requires sorted input.
Linear Search	Simple and works without sorting	Poorer scalability for large datasets.
Recursive Factorial	Compact recursive expression	Higher stack memory and recursion depth.
Iterative Factorial	Low auxiliary memory	Uses explicit iteration.
Single Loop	Efficient one-pass processing	Only suitable when one-pass logic is sufficient.
Nested Loop	Can process every pair/combination	Quadratic growth can become expensive.

7. Practical Selection Guide

Requirement	Recommended	Reason
Large sorted dataset with repeated lookups	Binary Search	Logarithmic search growth.
Small or unsorted collection	Linear Search	Simple and no sorting prerequisite.
Single factorial result	Iterative Factorial	$O(n)$ time with much lower auxiliary memory.
One-pass processing	Single Loop	Avoids unnecessary repeated work.
All-pair processing required	Nested Loop	Appropriate only when the problem actually requires pairwise work.

8. Conclusion

The project demonstrates that algorithm selection has a direct effect on scalability. The measured results broadly support the theoretical distinctions between $O(n)$, $O(\log n)$ and $O(n^2)$ behaviour. Small inputs can hide these differences, but the benchmark curves become increasingly separated as n grows.

- **Most efficient search implementation observed:** Binary Search for the sorted benchmark dataset.
- **Most space-efficient factorial implementation:** Iterative Factorial.
- **Most scalable loop implementation:** Single Loop.
- **Main experimental lesson:** growth rate matters more than a small-input timing advantage.
- **Main engineering lesson:** runtime, memory, input requirements and implementation constraints should be considered together.

9. Future Enhancements

- Add Jump Search and Interpolation Search.
- Allow users to benchmark their own safe code snippets.
- Add best-case and average-case dataset controls.
- Run more repetitions and report variability or confidence intervals.
- Export benchmark tables and graphs as CSV or PDF.
- Extend the tool with sorting algorithms.

10. Final Takeaway

The central lesson is that performance measurement should accompany theoretical analysis. A quadratic program may look acceptable at $n=100$ but become impractical as the input grows, while logarithmic and linear methods remain much more manageable. The tool makes these differences visible through repeatable measurements, operation counts, memory profiling and graphical comparison.

References listed in the submitted report: Horowitz, Sahni and Rajasekaran — Fundamentals of Computer Algorithms; Python documentation; Streamlit documentation; matplotlib documentation; memory profiling/tracemalloc documentation.

Supplementary Screenshot Evidence

This page consolidates the evidence list from the original final report that is not separately presented as a section in the three-part report. The actual Streamlit screenshots are already included in Part 1 (Figures A1–A4).

Figure	Evidence included in the final report
Figure A1 / Figure 1	Run Single output — Linear Search with input size 1000.
Figure A2 / Figure 2	Search Analysis output — Linear Search vs Binary Search.
Figure A3 / Figure 3	Code Benchmark output — comparison of Single Loop, Nested Loop, Recursive Factorial and Itera
Figure A4 / Figure 4	Automated Benchmark output — benchmark table and search performance graph.
Additional evidence	The report also contains the theoretical-vs-observed complexity table, benchmark tables, three perf

Report organization

Part 1 presents the project purpose, workflow, core modules, implementation evidence, experimental design and Streamlit screenshots. Part 2 presents the complete benchmark tables, graphs, trend analysis, benchmark summary and key findings. Part 3 presents theoretical-versus-observed complexity, algorithm comparisons, time–space trade-offs, practical selection guidance, conclusion and future enhancements.

Note: The final merged report uses the current benchmark tables and graphs from Parts 1–3. Older screenshots/tables with different timing values from the earlier draft were not reintroduced, so the final report remains internally consistent.